Week 4 Assignment

Statistics, Data Visualization & Exploratory Data Analysis

Internnova — Data Analyst Course (Python Track)

Prepared by: **Shruti Gupta**

Duration: 1 Week (7 Days) | Total Marks: 100

Submission format: Direct PDF Upload

This report applies statistical analysis, Matplotlib and Seaborn visualization, and exploratory data analysis (EDA) to a 12-order sales dataset — covering Tasks 1 through 9 of the assignment brief, with the full Python source code attached as an appendix.

Dataset Source

- Original file: `../week2_assignment_internnova/sales_data.csv`
- Raw dataset: 12 rows, 9 columns
- Cleaned dataset: 12 rows, 10 columns (adds a calculated **Sales** column)

Task 1: Mean, Median & Mode

(10 Marks)

- **Mean** of Units: 10.92
- **Median** of Units: 5.50
- **Mode** of Units: 5.00

Explanation: The mean shows the average unit count per order. The median shows the middle value and is useful when the data has a wide spread. The mode shows the most common order size in the dataset.

Task 2: Variance & Standard Deviation

(10 Marks)

- **Variance:** 192.63
- **Standard Deviation:** 13.88

Explanation: Standard deviation tells us how spread out the values are from the mean. A larger value means the order sizes vary more, while a smaller value means the numbers are closer together.

Task 3: Correlation & Probability Basics

(10 Marks)

Correlation

- Selected variables: **Price** and **Sales**
- Correlation coefficient: **0.68**
- Relationship: **positive (moderate positive)**

Probability

- Real-world example: probability that a randomly chosen order is from the North region
- North orders = 3 / Total orders = 12 = **0.25**
- As a percentage: **25.0%**

Explanation: The price-to-sales relationship is positive because as the value of the product rises, the sales revenue tends to rise as well. The probability example uses the share of orders from the North region, which is 3 out of 12 orders.

Task 4: Outlier Detection

(10 Marks)

- Method: Interquartile Range (IQR) on **Price**
- Lower bound: **-223.75** | Upper bound: **486.25**

OrderID	Product	Price ($)
1001	Laptop	750
1011	Laptop	750

Explanation: Outliers are unusually high or low values that can distort the mean and hide true patterns. They should be checked carefully, not automatically removed, because some outliers are valid premium transactions.

Task 5: Matplotlib Visualization

(15 Marks)

Five Matplotlib visualizations built from the cleaned sales dataset:

5.1 Line Chart

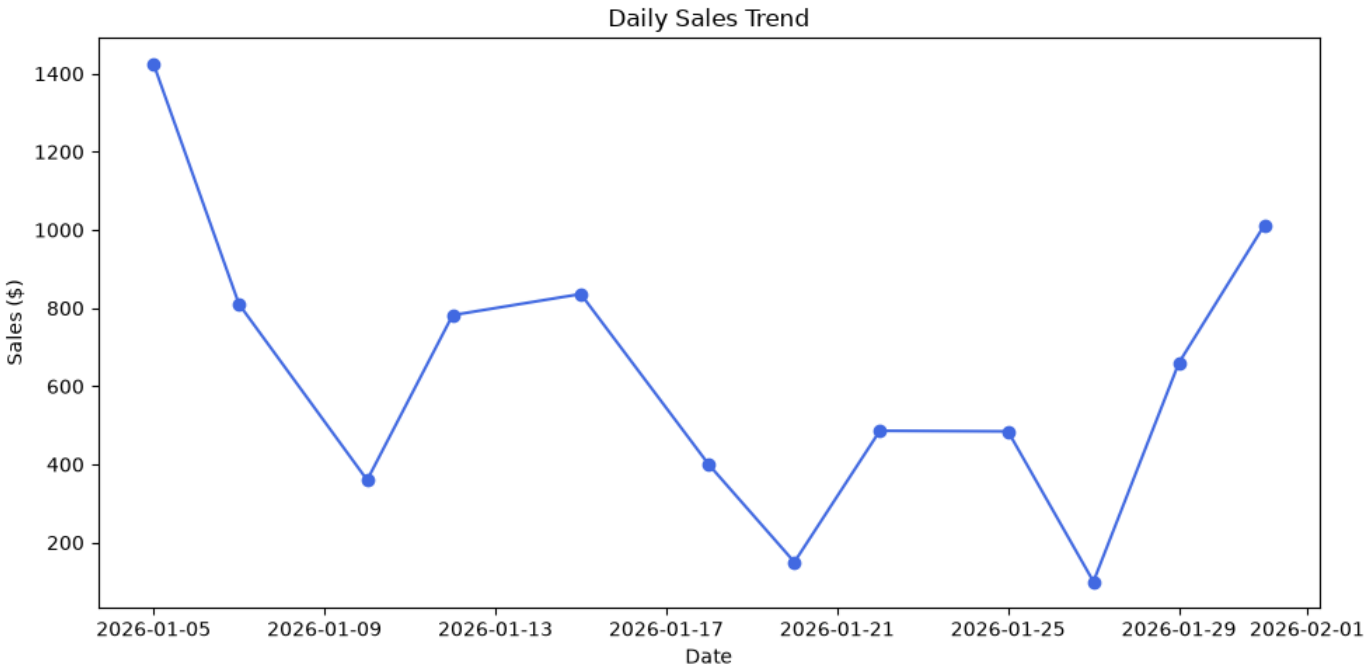


Figure 5.1 — Daily Sales Trend

Shows the daily sales trend over time and helps to see how sales change from one date to the next.

5.2 Bar Chart

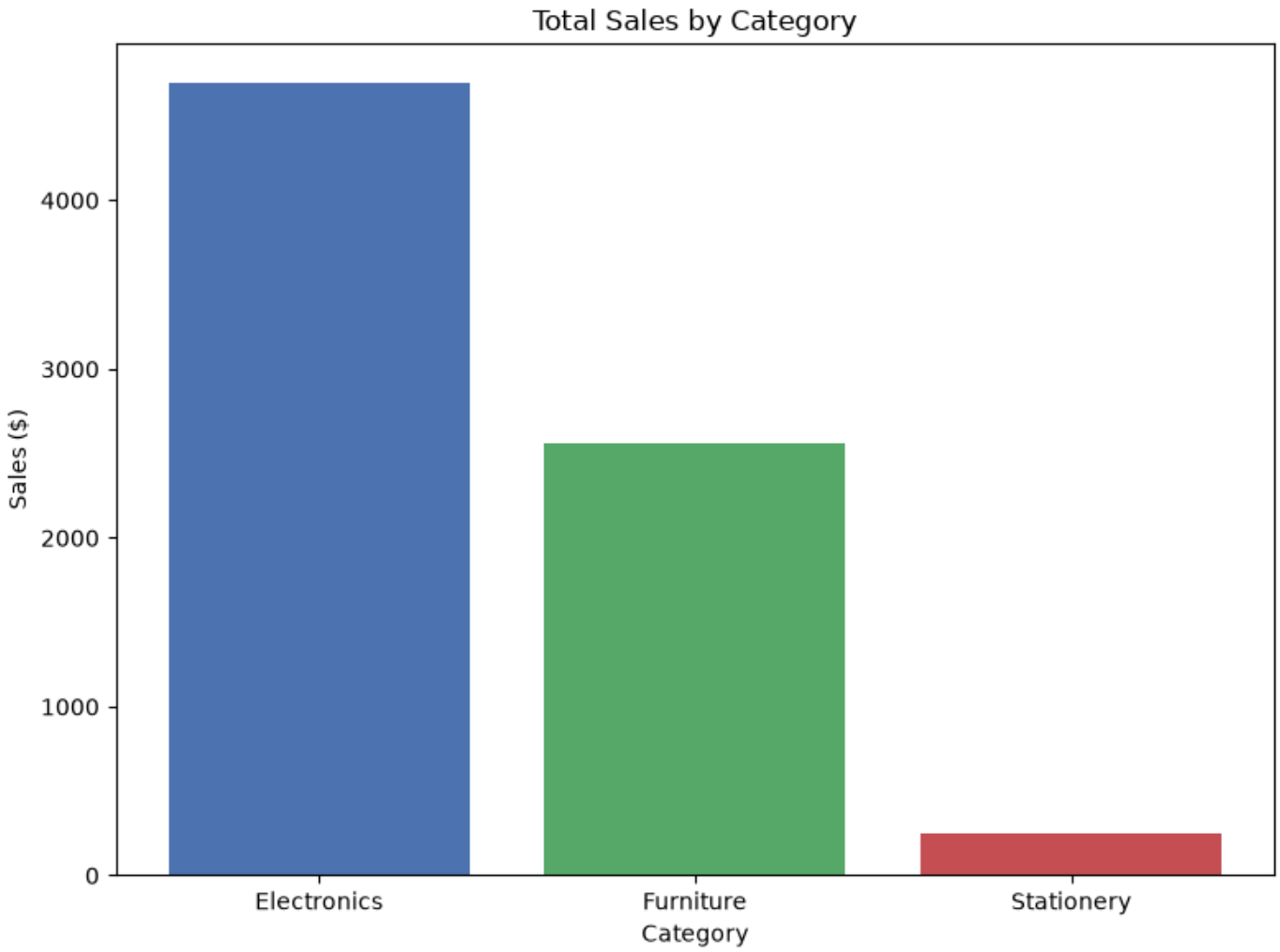


Figure 5.2 — Total Sales by Category

Compares total sales by category and shows which category contributes the most revenue.

5.3 Pie Chart

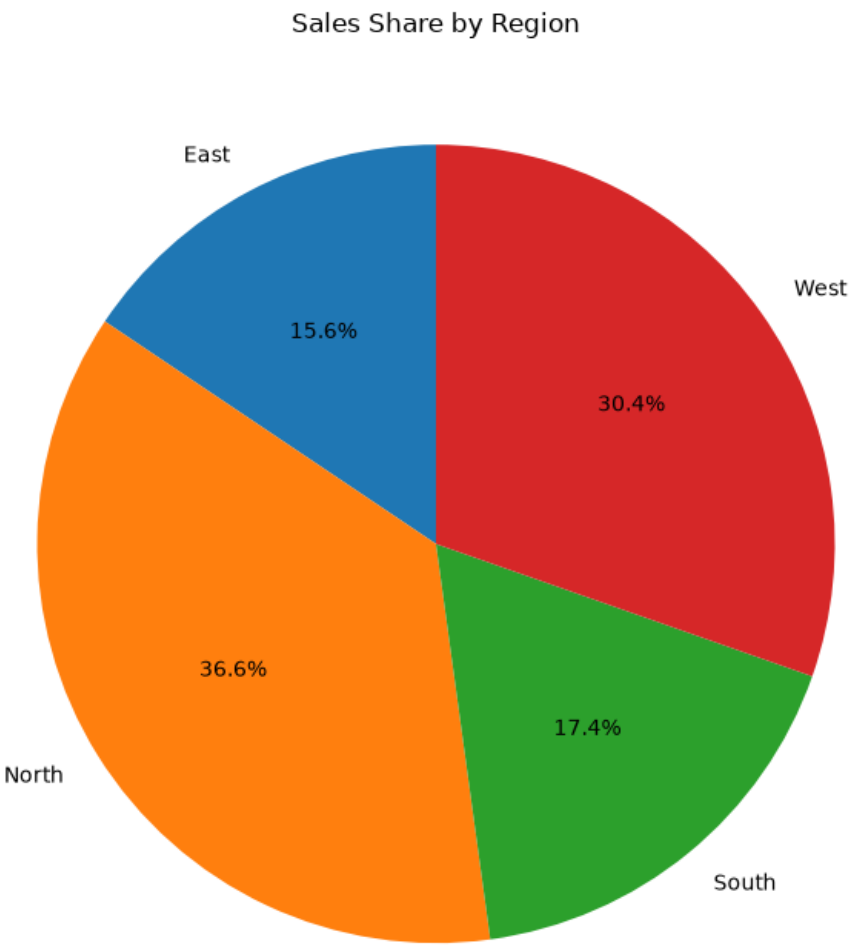


Figure 5.3 — Sales Share by Region

Displays the share of sales by region and makes regional contribution easy to compare.

5.4 Histogram

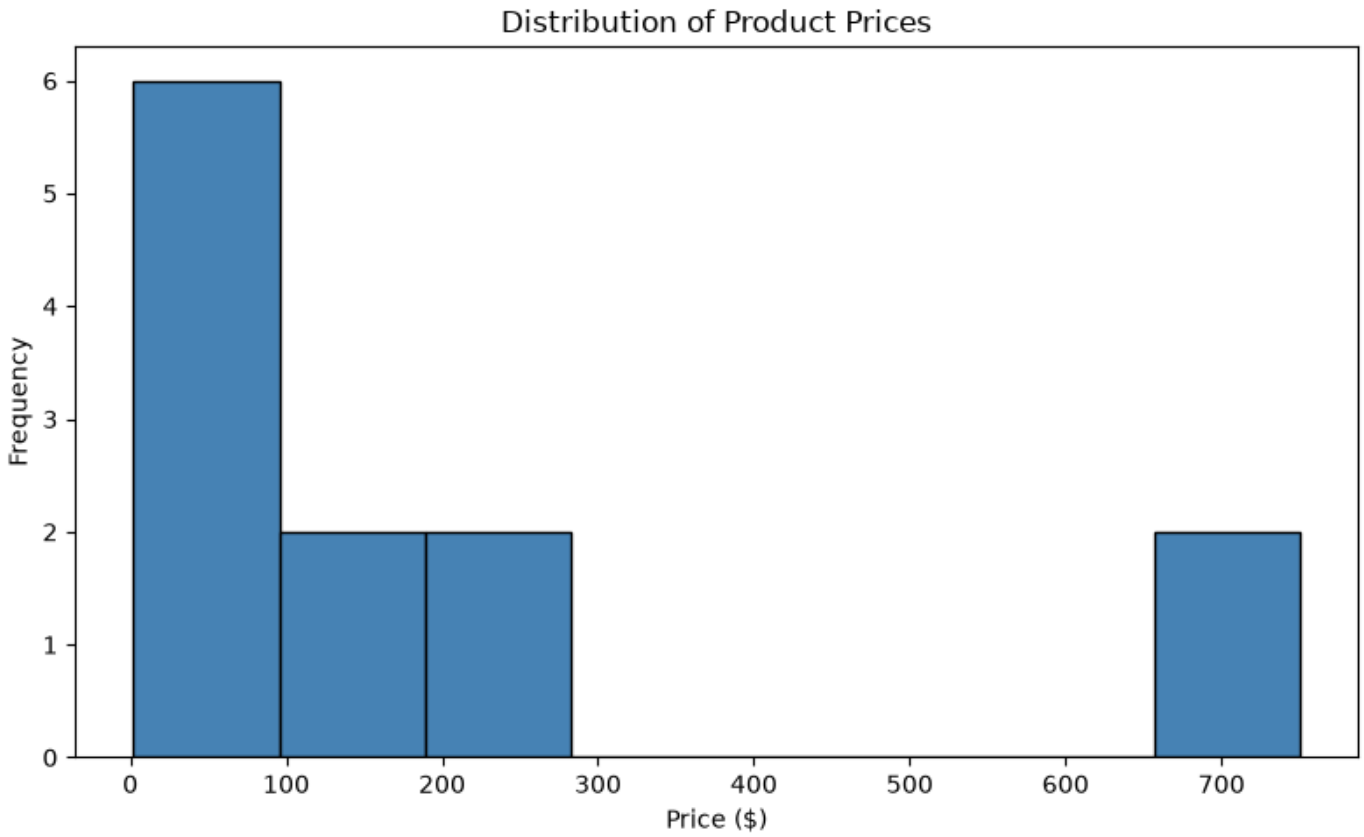


Figure 5.4 — Distribution of Product Prices

Shows the distribution of product prices and indicates whether prices are clustered or spread out.

5.5 Scatter Plot

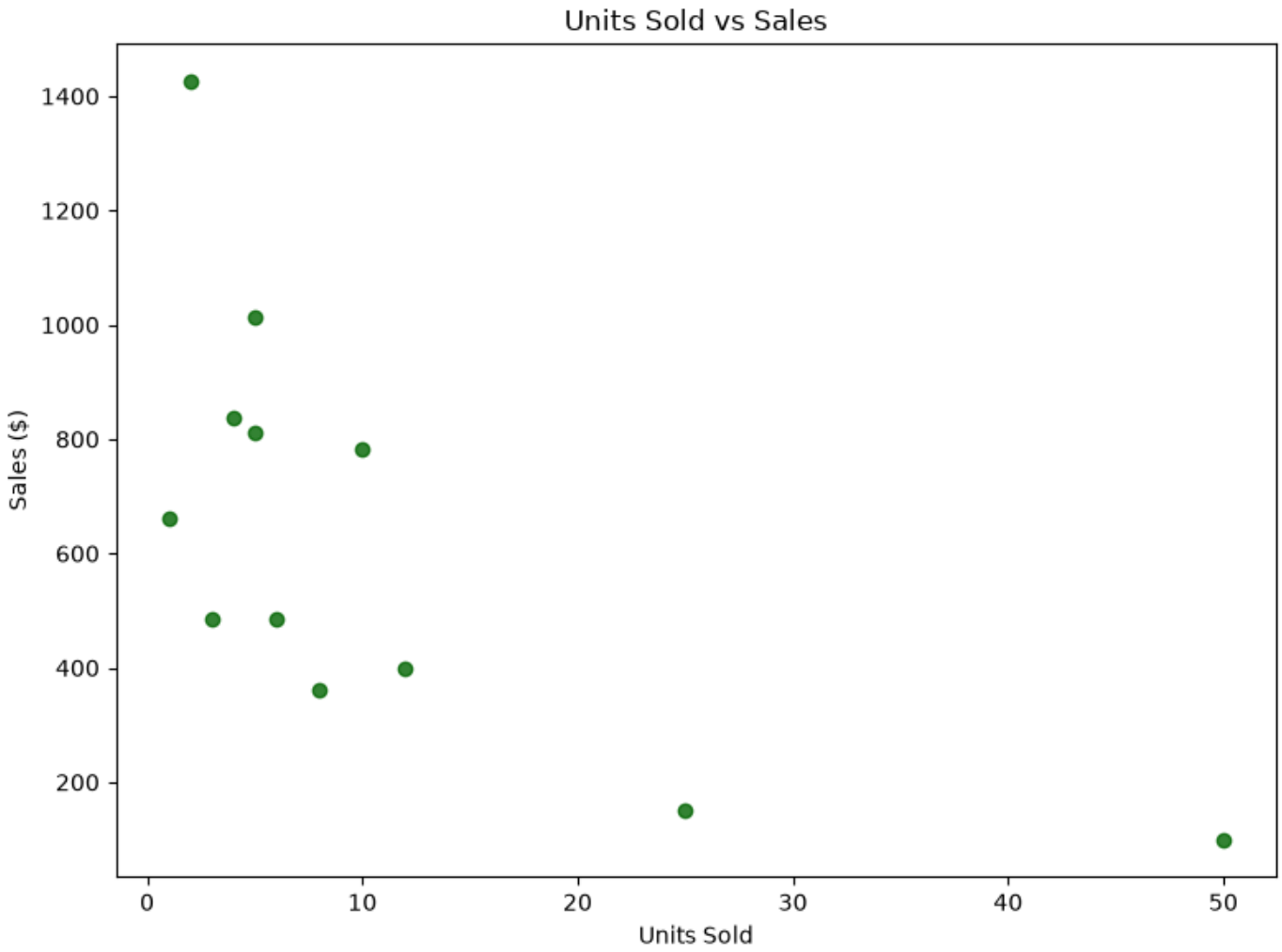


Figure 5.5 — Units Sold vs Sales

Compares units sold with sales and helps reveal whether larger order sizes are linked to higher revenue.

Task 6: Seaborn Visualization

(15 Marks)

Four Seaborn visualizations built from the cleaned sales dataset:

6.1 Count Plot

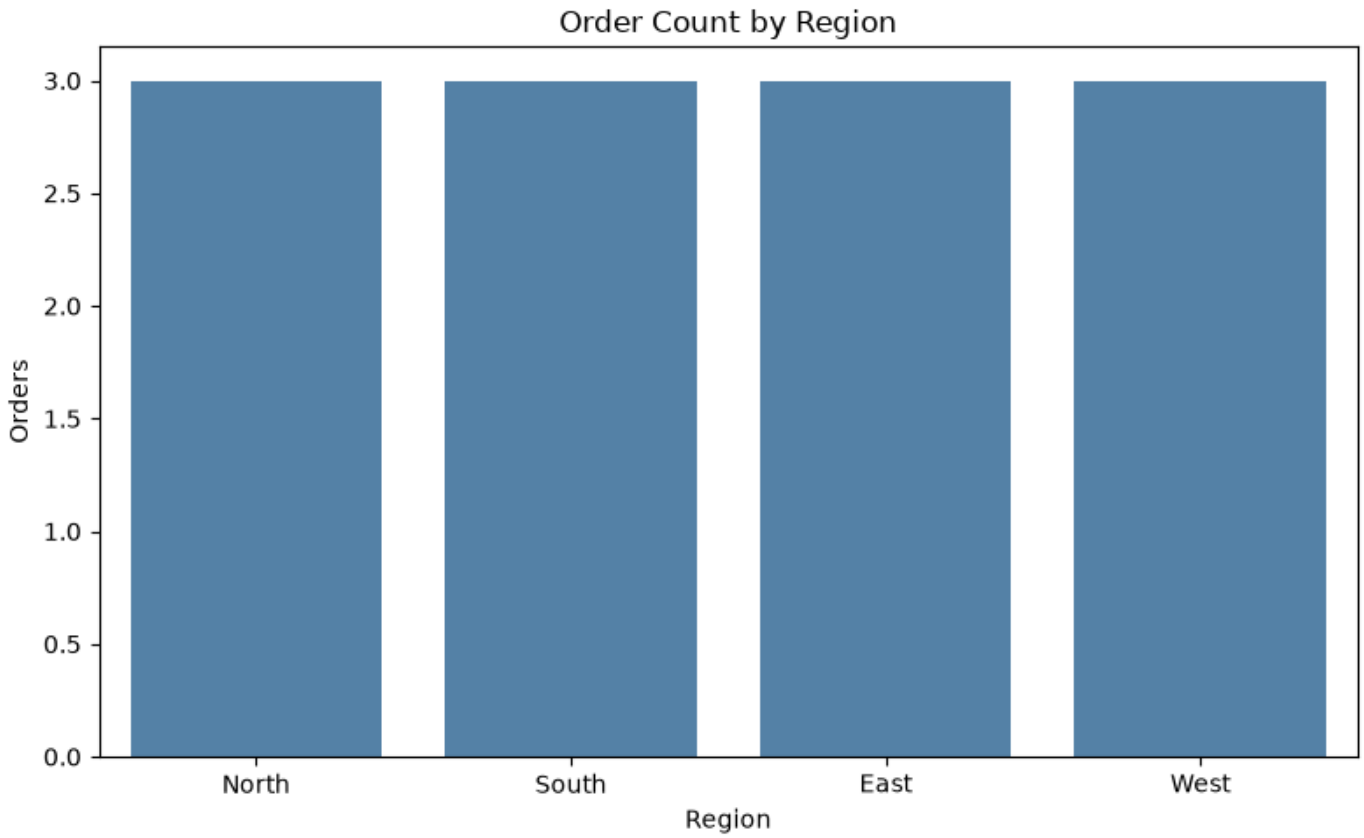


Figure 6.1 — Order Count by Region

Shows how many orders came from each region.

6.2 Box Plot

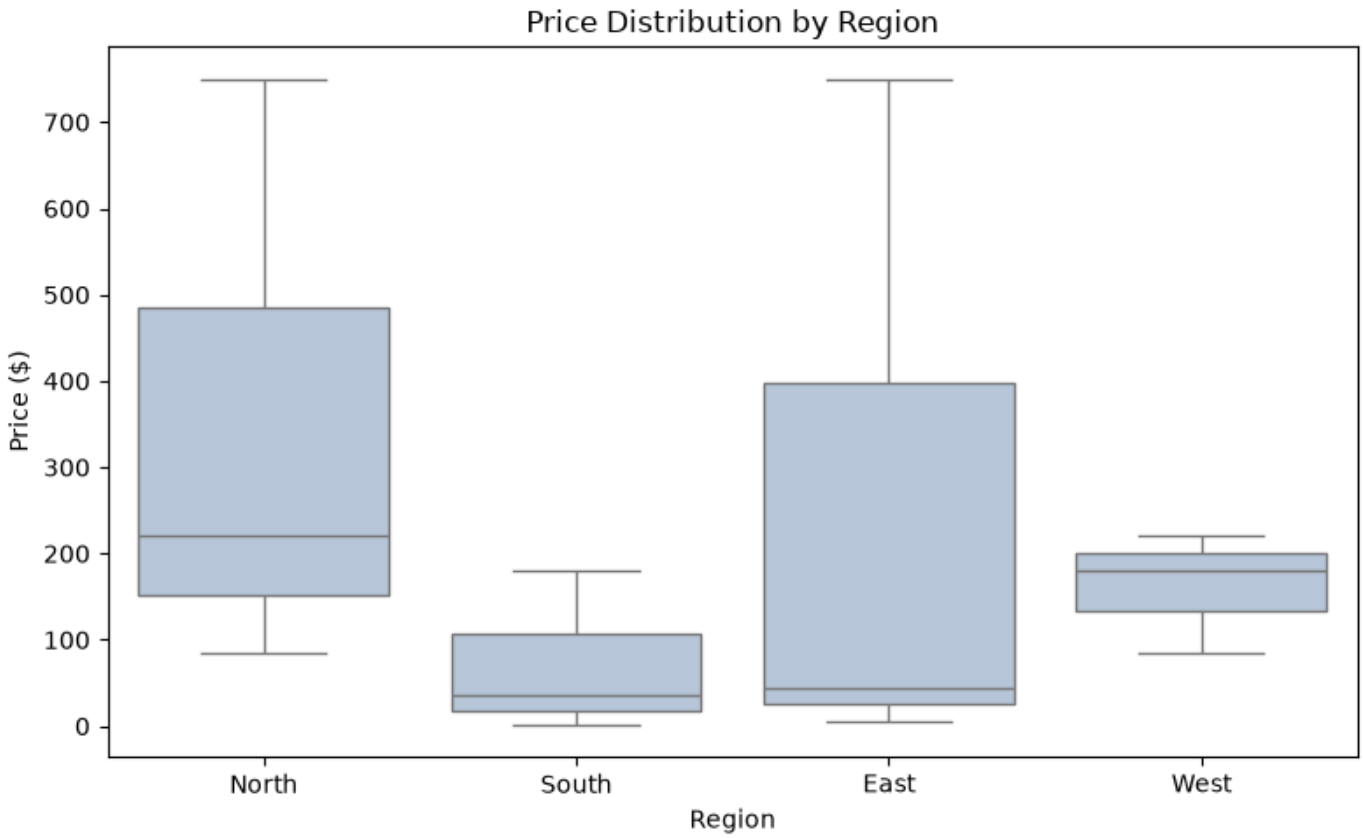


Figure 6.2 — Price Distribution by Region

Displays the spread and median of product prices by region.

6.3 Heatmap

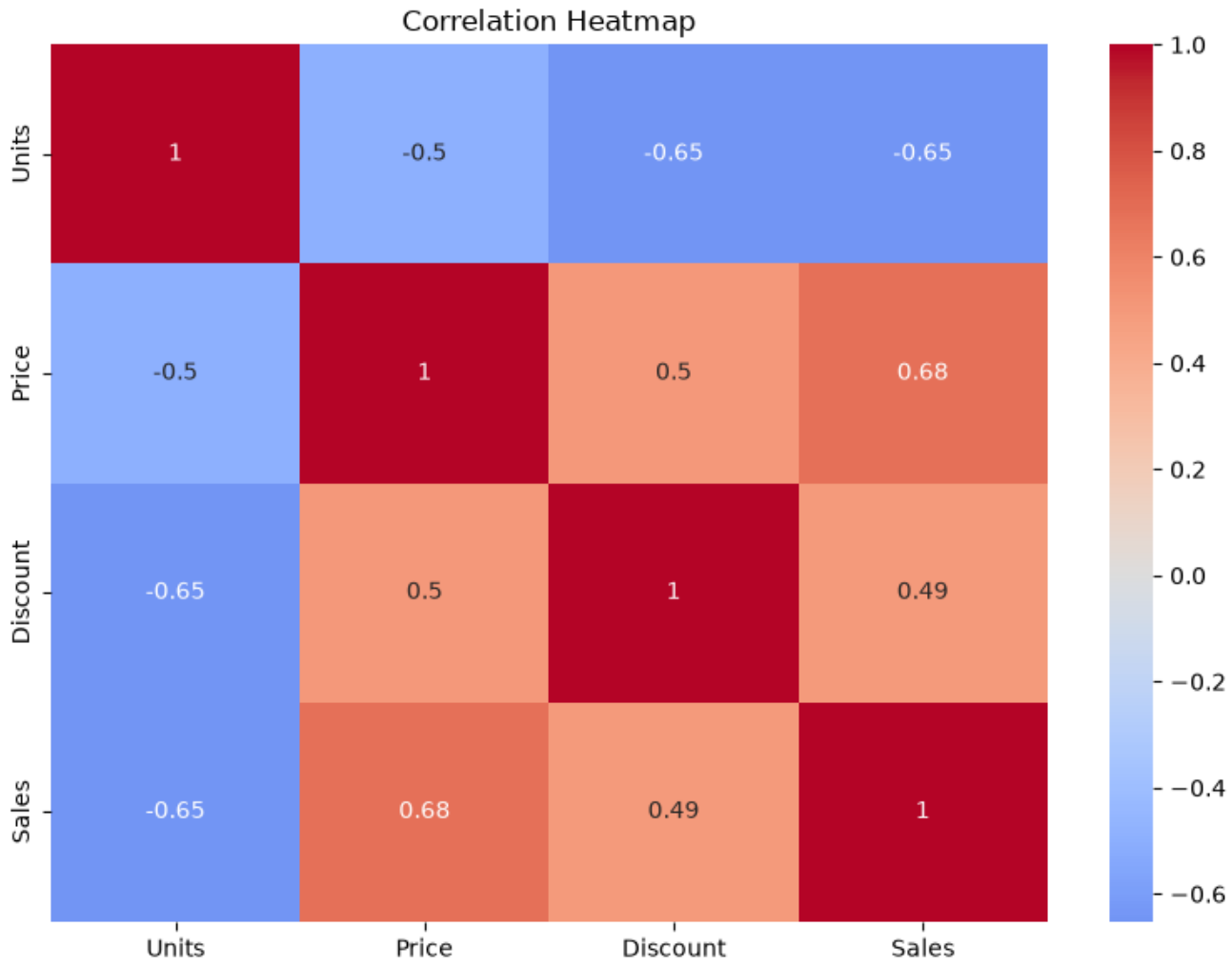


Figure 6.3 — Correlation Heatmap

Highlights the strength of relationships between numerical variables.

6.4 Pair Plot

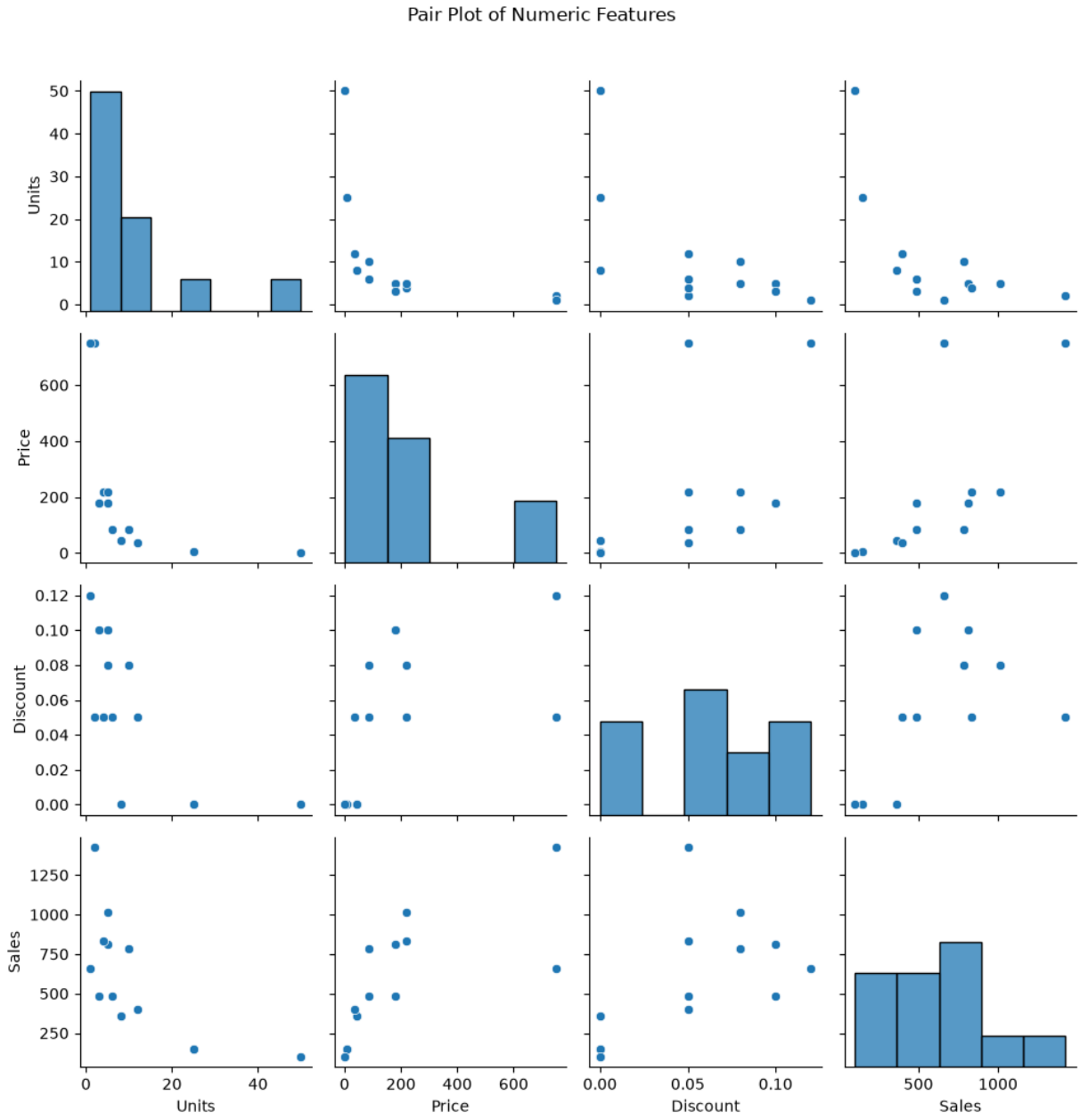


Figure 6.4 — Pair Plot of Numeric Features

Helps compare multiple variables at once to find patterns and clusters.

Task 7: EDA — Data Inspection & Cleaning

(15 Marks)

Data Inspection (Before Cleaning)

- Rows: 12 | Columns: 9
- Column names: OrderID, Date, Region, Product, Category, Units, Price, Salesperson, Discount
- Duplicate records: 0
- Incorrect / inconsistent values: the **Category** column had inconsistent text casing (e.g. “furniture” vs “Furniture”), and **Discount** had one missing value.

Column	Data Type
OrderID	int64
Date	str
Region	str
Product	str
Category	str
Units	int64
Price	int64
Salesperson	str
Discount	float64

Column	Missing Values
OrderID	0
Date	0
Region	0
Product	0
Category	0
Units	0
Price	0
Salesperson	0
Discount	1

Cleaning Steps

1. Trim extra spaces from text fields.
2. Standardize category names to a consistent format.
3. Convert the Date column to datetime format.
4. Fill the missing Discount value with the median discount value.

5 Create a Sales column from Units × Price × (1 − Discount).

6 Remove duplicate rows if any are found.

Data Inspection (After Cleaning)

- Rows: 12 | Columns: 10 (adds the calculated **Sales** column)
- Missing values: **0** across all 10 columns
- Duplicate records: **0**
- Saved to `cleaned_sales_data.csv` and `dataset_after_cleaning.csv`

Statistical Summary (cleaned data)

	Units	Price	Discount	Sales
count	12.00	12.00	12.00	12.00
mean	10.92	213.17	0.06	625.38
std	13.88	262.69	0.04	376.10
min	1.00	2.00	0.00	100.00
25%	3.75	42.50	0.04	389.25
50%	5.50	132.50	0.05	573.00
75%	10.50	220.00	0.08	816.50
max	50.00	750.00	0.12	1425.00

Screenshots: Before vs. After Cleaning

Dataset Before Cleaning

OrderID	Date	Region	Product	Category	Units	Price	Salesperson	Discount
1001	2026-01-05	North	Laptop	Electronics	2	750	Asha	0.05
1002	2026-01-07	South	Desk	furniture	5	180	Ravi	0.1
1003	2026-01-10	East	Headphones	Electronics	8	45	Meera	0.0
1004	2026-01-12	West	Chair	Furniture	10	85	Arjun	0.08
1005	2026-01-15	North	Monitor	Electronics	4	220	Asha	nan
1006	2026-01-18	South	Keyboard	Electronics	12	35	Ravi	0.05
1007	2026-01-20	East	Notebook	Stationery	25	6	Meera	0.0
1008	2026-01-22	West	Desk	Furniture	3	180	Arjun	0.1
1009	2026-01-25	North	Chair	Furniture	6	85	Asha	0.05
1010	2026-01-27	South	Pen	Stationery	50	2	Ravi	0.0
1011	2026-01-29	East	Laptop	Electronics	1	750	Meera	0.12
1012	2026-01-31	West	Monitor	Electronics	5	220	Arjun	0.08

Figure 7.1 — Dataset Before Cleaning

Dataset After Cleaning

OrderID	Date	Region	Product	Category	Units	Price	Salesperson	Discount	Sales
1001	2026-01-05 00:00:00	North	Laptop	Electronics	2	750	Asha	0.05	1425.0
1002	2026-01-07 00:00:00	South	Desk	Furniture	5	180	Ravi	0.1	810.0
1003	2026-01-10 00:00:00	East	Headphones	Electronics	8	45	Meera	0.0	360.0
1004	2026-01-12 00:00:00	West	Chair	Furniture	10	85	Arjun	0.08	782.0
1005	2026-01-15 00:00:00	North	Monitor	Electronics	4	220	Asha	0.05	836.0
1006	2026-01-18 00:00:00	South	Keyboard	Electronics	12	35	Ravi	0.05	399.0
1007	2026-01-20 00:00:00	East	Notebook	Stationery	25	6	Meera	0.0	150.0
1008	2026-01-22 00:00:00	West	Desk	Furniture	3	180	Arjun	0.1	486.0
1009	2026-01-25 00:00:00	North	Chair	Furniture	6	85	Asha	0.05	484.5
1010	2026-01-27 00:00:00	South	Pen	Stationery	50	2	Ravi	0.0	100.0
1011	2026-01-29 00:00:00	East	Laptop	Electronics	1	750	Meera	0.12	660.0
1012	2026-01-31 00:00:00	West	Monitor	Electronics	5	220	Arjun	0.08	1012.0

Figure 7.2 — Dataset After Cleaning

Task 8: EDA — Correlation & Insights

(10 Marks)

The numerical correlation matrix (Figure 6.3, Correlation Heatmap) shows that **Price and Sales** have a positive relationship (**0.68**), while **Units and Sales** have a negative relationship (**-0.65**).

- Products with higher prices can generate more sales revenue even when the order size is not the highest.
- Discounting has a mild relationship with sales, which suggests that discount strategy should be reviewed carefully and not assumed to improve revenue automatically.

Key Insights

1. Price is a major factor in sales revenue, so premium products matter.
2. Unit volume alone does not guarantee higher revenue because product pricing varies substantially.
3. Discounting should be managed strategically, since the effect on revenue is not always positive.

Task 9: Business Recommendations

(5 Marks)

1. Focus marketing and stock attention on higher-priced products with strong sales potential, because the data shows a positive relationship between price and sales revenue.
2. Review discount decisions carefully, especially for premium products, because the dataset shows that discounting changes revenue and should not be applied without checking the sales effect.
3. Use the outlier checks to monitor unusual premium orders before making pricing changes, so valid high-value sales do not get mistaken for data problems.

Conclusion

The original Week 2 sales dataset was inspected, cleaned, and analyzed using statistics, visualizations, and EDA methods. The results show that price, category mix, and discount strategy all influence sales outcomes. The project keeps the analysis simple, transparent, and aligned with the assignment rubric.

Appendix A — Full Python Source Code

week4_statistics_eda.py — the complete script used to clean the data, compute every statistic above, generate all nine chart images, and write this report's source data.

```
from __future__ import annotations

from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

ROOT = Path(__file__).resolve().parent
RAW_DATA_PATH = ROOT.parent / "week2_assignment_internnova" / "sales_data.csv"
OUTPUT_DIR = ROOT / "outputs"
OUTPUT_DIR.mkdir(exist_ok=True)
REPORT_PATH = ROOT / "WEEK4_REPORT.md"
BEFORE_CSV_PATH = ROOT / "dataset_before_cleaning.csv"
AFTER_CSV_PATH = ROOT / "dataset_after_cleaning.csv"
CLEANED_CSV_PATH = ROOT / "cleaned_sales_data.csv"


def verify_raw_dataset() -> pd.DataFrame:
    if not RAW_DATA_PATH.exists():
        raise FileNotFoundError(
            f"Missing original dataset: {RAW_DATA_PATH}. "
            "Please check the sibling folder /python-for-ai/week2_assignment_internnova/sales_data.csv."
        )
    return pd.read_csv(RAW_DATA_PATH)


def save_table_snapshot(df: pd.DataFrame, png_path: Path, title: str) -> None:
    fig, ax = plt.subplots(figsize=(14, max(4.0, 0.7 * (len(df) + 2))))
    ax.axis("off")
    table = ax.table(
        cellText=df.head(12).values,
        colLabels=df.columns,
        loc="center",
        cellLoc="center",
    )
    table.auto_set_font_size(False)
    table.set_fontsize(8)
    table.scale(1.15, 1.6)
    ax.set_title(title, fontsize=12, pad=10)
    fig.tight_layout()
    fig.savefig(png_path, dpi=200, bbox_inches="tight")
    plt.close(fig)


def clean_sales_data(raw_df: pd.DataFrame) -> pd.DataFrame:
    df = raw_df.copy()
    df.columns = [col.strip() for col in df.columns]

    for col in ["Region", "Product", "Category", "Salesperson"]:
        df[col] = df[col].astype(str).str.strip()

    df["Region"] = df["Region"].str.title()
    df["Product"] = df["Product"].str.title()
    df["Category"] = df["Category"].str.title()
    df["Salesperson"] = df["Salesperson"].str.title()

    df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
    df["Discount"] = pd.to_numeric(df["Discount"], errors="coerce")
```

```
    df["Discount"] = df["Discount"].fillna(df["Discount"].median()).round(2)
    df["Sales"] = (df["Units"] * df["Price"] * (1 - df["Discount"])).round(2)

    df = df.drop_duplicates().reset_index(drop=True)
    return df


def task_1_mean_median_mode(df: pd.DataFrame) -> dict:
    units = df["Units"]
    mode_value = units.mode()
    return {
        "mean": float(units.mean()),
        "median": float(units.median()),
        "mode": float(mode_value.iloc[0]) if not mode_value.empty else float("nan"),
    }


def task_2_variance_std(df: pd.DataFrame) -> dict:
    units = df["Units"]
    return {
        "variance": float(units.var(ddof=1)),
        "std_dev": float(units.std(ddof=1)),
    }


def task_3_correlation_probability(df: pd.DataFrame) -> dict:
    corr = df["Price"].corr(df["Sales"])
    north_count = int((df["Region"] == "North").sum())
    total_orders = len(df)
    prob_value = north_count / total_orders if total_orders else 0.0
    return {
        "correlation_price_sales": float(corr),
        "relationship": "positive" if corr > 0 else "negative" if corr < 0 else "weak/neutral",
        "north_count": north_count,
        "total_orders": total_orders,
        "north_probability": float(prob_value),
        "north_percentage": float(prob_value * 100),
    }


def task_4_outliers(df: pd.DataFrame) -> pd.DataFrame:
    q1 = df["Price"].quantile(0.25)
    q3 = df["Price"].quantile(0.75)
    iqr = q3 - q1
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr
    return df[(df["Price"] < lower_bound) | (df["Price"] > upper_bound)].copy()


def save_matplotlib_visuals(df: pd.DataFrame) -> None:
    # 1. Line chart
    daily_sales = df.groupby(df["Date"].dt.date)["Sales"].sum()
    plt.figure(figsize=(10, 5))
    plt.plot(daily_sales.index, daily_sales.values, marker="o", color="royalblue")
    plt.title("Daily Sales Trend")
    plt.xlabel("Date")
    plt.ylabel("Sales ($)")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "matplotlib_line_chart.png")
    plt.close()

    # 2. Bar chart
    category_sales = df.groupby("Category")["Sales"].sum().sort_values(ascending=False)
    plt.figure(figsize=(8, 6))
    plt.bar(category_sales.index, category_sales.values, color=["#4C72B0", "#55A868", "#C44E52"])
    plt.title("Total Sales by Category")
    plt.xlabel("Category")
```

```
    plt.ylabel("Sales ($)")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "matplotlib_bar_chart.png")
    plt.close()

    # 3. Pie chart
    region_sales = df.groupby("Region")["Sales"].sum()
    plt.figure(figsize=(7, 7))
    plt.pie(region_sales.values, labels=region_sales.index, autopct="%1.1f%%", startangle=90)
    plt.title("Sales Share by Region")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "matplotlib_pie_chart.png")
    plt.close()

    # 4. Histogram
    plt.figure(figsize=(8, 5))
    plt.hist(df["Price"], bins=8, color="steelblue", edgecolor="black")
    plt.title("Distribution of Product Prices")
    plt.xlabel("Price ($)")
    plt.ylabel("Frequency")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "matplotlib_histogram.png")
    plt.close()

    # 5. Scatter plot
    plt.figure(figsize=(8, 6))
    plt.scatter(df["Units"], df["Sales"], color="darkgreen", alpha=0.8)
    plt.title("Units Sold vs Sales")
    plt.xlabel("Units Sold")
    plt.ylabel("Sales ($)")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "matplotlib_scatter_plot.png")
    plt.close()


def save_seaborn_visuals(df: pd.DataFrame) -> None:
    numeric_df = df[["Units", "Price", "Discount", "Sales"]]

    plt.figure(figsize=(8, 5))
    sns.countplot(data=df, x="Region", color="steelblue")
    plt.title("Order Count by Region")
    plt.xlabel("Region")
    plt.ylabel("Orders")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "seaborn_countplot.png")
    plt.close()

    plt.figure(figsize=(8, 5))
    sns.boxplot(data=df, x="Region", y="Price", color="lightsteelblue")
    plt.title("Price Distribution by Region")
    plt.xlabel("Region")
    plt.ylabel("Price ($)")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "seaborn_boxplot.png")
    plt.close()

    corr = numeric_df.corr()
    plt.figure(figsize=(8, 6))
    sns.heatmap(corr, annot=True, cmap="coolwarm", center=0)
    plt.title("Correlation Heatmap")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "seaborn_heatmap.png")
    plt.close()

    pair_plot = sns.pairplot(numeric_df, diag_kind="hist")
    pair_plot.fig.suptitle("Pair Plot of Numeric Features", y=1.02)
    pair_plot.fig.tight_layout()
```

```
    pair_plot.savefig(OUTPUT_DIR / "seaborn_pairplot.png")
    plt.close(pair_plot.fig)


def generate_report(raw_df: pd.DataFrame, cleaned_df: pd.DataFrame, stats: dict, corr_prob: dict,
    outliers: pd.DataFrame) -> None:
    before_missing = raw_df.isna().sum().to_dict()
    before_duplicates = int(raw_df.duplicated().sum())
    after_missing = cleaned_df.isna().sum().to_dict()
    after_duplicates = int(cleaned_df.duplicated().sum())
    numeric_summary = cleaned_df[["Units", "Price", "Discount", "Sales"]].describe().round(2).to_string()

    lines = [
        "# Week 4 Assignment: Statistics, Data Visualization & EDA",
        "",
        "## Dataset Source",
        "- Original file: ../week2_assignment_internnova/sales_data.csv",
        "- Raw rows: 12",
        "- Raw columns: 9",
        "- Cleaned rows: " + str(len(cleaned_df)),
        "- Cleaned columns: " + str(len(cleaned_df.columns)),
        "",
        "## Task 1: Mean, Median & Mode",
        f"- Mean of Units: {stats['mean']:.2f}",
        f"- Median of Units: {stats['median']:.2f}",
        f"- Mode of Units: {stats['mode']:.2f}",
        "- Explanation: The mean shows the average unit count per order. The median shows the middle value
            and is useful when the data has a wide spread. The mode shows the most common order size in the
            dataset.",
        "",
        "## Task 2: Variance & Standard Deviation",
        f"- Variance: {stats['variance']:.2f}",
        f"- Standard Deviation: {stats['std_dev']:.2f}",
        "- Explanation: Standard deviation tells us how spread out the values are from the mean. A larger
            value means the order sizes vary more, while a smaller value means the numbers are closer
            together.",
        "",
        "## Task 3: Correlation & Probability Basics",
        f"- Selected variables: Price and Sales",
        f"- Correlation coefficient: {corr_prob['correlation_price_sales']:.2f}",
        f"- Relationship: {corr_prob['relationship']} (moderate positive)",
        f"- Probability example: North orders = {corr_prob['north_count']} / Total orders =
            {corr_prob['total_orders']} = {corr_prob['north_probability']:.2f}",
        f"- Percentage: {corr_prob['north_percentage']:.1f}%",
        "- Explanation: The price-to-sales relationship is positive because as the value of the product
            rises, the sales revenue tends to rise as well. The probability example uses the share of orders
            from the North region, which is 3 out of 12 orders.",
        "",
        "## Task 4: Outlier Detection",
        f"- IQR method lower bound: {cleaned_df['Price'].quantile(0.25) - 1.5 *
            (cleaned_df['Price'].quantile(0.75) - cleaned_df['Price'].quantile(0.25)):.2f}",
        f"- IQR method upper bound: {cleaned_df['Price'].quantile(0.75) + 1.5 *
            (cleaned_df['Price'].quantile(0.75) - cleaned_df['Price'].quantile(0.25)):.2f}",
        f"- Identified outliers: {outliers[['OrderID', 'Product', 'Price']].to_dict(orient='records')}",
        "- Explanation: Outliers are unusually high or low values that can distort the mean and hide true
            patterns. They should be checked carefully, not automatically removed, because some outliers are
            valid premium transactions.",
        "",
        "## Task 5: Matplotlib Visualization",
        "- Line chart: shows the daily sales trend over time and helps to see how sales change from one
            date to the next.",
        "- Bar chart: compares total sales by category and shows which category contributes the most
            revenue.",
        "- Pie chart: displays the share of sales by region and makes regional contribution easy to
            compare.",
        "- Histogram: shows the distribution of product prices and indicates whether prices are clustered
            or spread out.",
```

```
"- Scatter plot: compares units sold with sales and helps reveal whether larger order sizes are
    linked to higher revenue.",
"",
"## Task 6: Seaborn Visualization",
"- Count plot: shows how many orders came from each region.",
"- Box plot: displays the spread and median of product prices by region.",
"- Heatmap: highlights the strength of relationships between numerical variables.",
"- Pair plot: helps compare multiple variables at once to find patterns and clusters.",
"",
"## Task 7: EDA — Data Inspection & Cleaning",
"### Before cleaning",
f"- Rows: {len(raw_df)}",
f"- Columns: {len(raw_df.columns)}",
f"- Column names: {list(raw_df.columns)}",
f"- Data types: {raw_df.dtypes.astype(str).to_dict()}",
f"- Missing values: {before_missing}",
f"- Duplicate records: {before_duplicates}",
"- Incorrect/inconsistent values: Category had inconsistent text and the Discount column had one
    missing value.",
"",
"### Cleaning steps",
"1. Trim extra spaces from text fields.",
"2. Standardize category names to a consistent format.",
"3. Convert the Date column to datetime format.",
"4. Fill the missing Discount value with the median discount value.",
"5. Create a Sales column from Units × Price × (1 − Discount).",
"6. Remove duplicate rows if any are found.",
"",
"### After cleaning",
f"- Rows: {len(cleaned_df)}",
f"- Columns: {len(cleaned_df.columns)}",
f"- Missing values: {after_missing}",
f"- Duplicate records: {after_duplicates}",
"- The cleaned dataset was saved to cleaned_sales_data.csv and dataset_after_cleaning.csv.",
"",
"### Data Summary",
"```text",
numeric_summary,
"```",
"",
"- Before/after cleaning evidence is saved as dataset_before_cleaning.png and
    dataset_after_cleaning.png in the project folder.",
"",
"## Task 8: EDA — Correlation & Insights",
"- The numerical correlation matrix shows that Price and Sales have a positive relationship
    (0.68), while Units and Sales have a negative relationship (-0.65).",
"- Products with higher prices can generate more sales revenue even when the order size is not the
    highest.",
"- Discounting has a mild relationship with sales, which suggests that discount strategy should be
    reviewed carefully and not assumed to improve revenue automatically.",
"- Insight 1: Price is a major factor in sales revenue, so premium products matter.",
"- Insight 2: Unit volume alone does not guarantee higher revenue because product pricing varies
    substantially.",
"- Insight 3: Discounting should be managed strategically, since the effect on revenue is not
    always positive.",
"",
"## Task 9: Business Recommendations",
"1. Focus marketing and stock attention on higher-priced products with strong sales potential,
    because the data shows a positive relationship between price and sales revenue.",
"2. Review discount decisions carefully, especially for premium products, because the dataset
    shows that discounting changes revenue and should not be applied without checking the sales
    effect.",
"3. Use the outlier checks to monitor unusual premium orders before making pricing changes, so
    valid high-value sales do not get mistaken for data problems.",
"",
"## Conclusion",
"The original Week 2 sales dataset was inspected, cleaned, and analyzed using statistics,
```

```
            visualizations, and EDA methods. The results show that price, category mix, and discount strategy
            all influence sales outcomes. The project keeps the analysis simple, transparent, and aligned with
            the assignment rubric.",
        ]
        REPORT_PATH.write_text("\n".join(lines), encoding="utf-8")


    def main() -> None:
        raw_df = verify_raw_dataset()
        BEFORE_CSV_PATH.write_text(raw_df.to_csv(index=False), encoding="utf-8")
        save_table_snapshot(raw_df, ROOT / "dataset_before_cleaning.png", "Dataset Before Cleaning")

        cleaned_df = clean_sales_data(raw_df)
        cleaned_df.to_csv(AFTER_CSV_PATH, index=False)
        cleaned_df.to_csv(CLEANED_CSV_PATH, index=False)
        save_table_snapshot(cleaned_df, ROOT / "dataset_after_cleaning.png", "Dataset After Cleaning")

        stats = task_1_mean_median_mode(cleaned_df)
        stats.update(task_2_variance_std(cleaned_df))
        corr_prob = task_3_correlation_probability(cleaned_df)
        outliers = task_4_outliers(cleaned_df)

        save_matplotlib_visuals(cleaned_df)
        save_seaborn_visuals(cleaned_df)
        generate_report(raw_df, cleaned_df, stats, corr_prob, outliers)

        print("Original dataset found:", RAW_DATA_PATH)
        print("Raw rows:", len(raw_df), "Raw columns:", len(raw_df.columns))
        print("Raw column names:", list(raw_df.columns))
        print("Raw data types:", raw_df.dtypes.astype(str).to_dict())
        print("Cleaned rows:", len(cleaned_df), "Cleaned columns:", len(cleaned_df.columns))
        print("Mean Units:", round(stats["mean"], 2))
        print("Median Units:", round(stats["median"], 2))
        print("Mode Units:", round(stats["mode"], 2))
        print("Variance:", round(stats["variance"], 2))
        print("Standard Deviation:", round(stats["std_dev"], 2))
        print("Correlation Price-Sales:", round(corr_prob["correlation_price_sales"], 4))
        print("North probability:", round(corr_prob["north_probability"], 4))
        print("Outlier records:")
        print(outliers[["OrderID", "Product", "Price"]])
        print("Files saved to:", ROOT)


    if __name__ == "__main__":
        main()
```